

Spring Security OAuth 2.0 Lab

Authorization Server + Resource Server

Scenario

You already have the Order Management API from the previous exercise. In this lab, you will split security responsibilities across two applications. A dedicated **Authorization Server** authenticates users, manages OAuth 2.0 clients, and issues access tokens. The existing Order Management API becomes a **Resource Server** that accepts and validates those tokens.

Authorization Server	Access Token (JWT)	Order API
Users + registered clients stored in a database	Signed token carrying scopes and user authorities	Existing application converted to a Resource Server

Learning objectives

- Build a Spring Authorization Server and understand the responsibilities of its core components.
- Persist end-user credentials and OAuth 2.0 registered client information in a relational database.
- Issue signed JWT access tokens that contain the information required by downstream authorization rules.
- Convert an existing Spring Security application from HTTP Basic authentication to OAuth 2.0 Resource Server authentication.
- Preserve endpoint-level and method-level authorization while changing the authentication mechanism.

Important constraint

Do not remove the authorization rules from the previous exercise. The goal is to change **where authentication happens** and **how the API receives the authenticated principal**, while keeping authorization behavior equivalent.

TASK 1

Implement an Authorization Server

Persist users and registered OAuth 2.0 clients in a database

Create a separate Spring Boot application that acts as the Authorization Server.

1.1 Configure Spring Authorization Server

- Add the Spring Authorization Server support and configure the required security filter chains.
- Run the Authorization Server on a different port from the Order API, for example **9000**.
- Configure token signing so the server issues signed JWT access tokens.
- Expose the standard OAuth 2.0 metadata and JWK endpoints required by a Resource Server.

1.2 Store users in the database

Replace in-memory users with database-backed user authentication. Create an appropriate schema/entity model and a **UserDetailsService** (or equivalent) that loads users from the database.

Username	Password	Roles / Authorities
alice	password	CUSTOMER
bob	password	CUSTOMER, ORDER_REFUND
manager	password	MANAGER
admin	password	ADMIN

Passwords must be encoded using a **PasswordEncoder**. Do not store plain-text passwords in the database.

1.3 Store registered clients in the database

Configure the Authorization Server so registered client details are no longer stored in memory. Use **JdbcRegisteredClientRepository** or an equivalent persistent implementation.

- Create the database schema required for registered clients.
- Insert at least one client that can perform the **authorization_code** flow.
- Configure a client ID, encoded client secret, redirect URI, allowed grant types, and scopes.
- Allow **refresh_token** as an additional grant type.
- Choose scopes that will later be useful to the Order API, for example **orders.read** and **orders.write**.

Implementation hint

Spring Authorization Server provides a JDBC implementation named **JdbcRegisteredClientRepository**. It expects the corresponding registered-client table to exist in the database. You may instead implement persistence yourself with JPA if you want the extra challenge.

1.4 Include user authorities in the access token

The Resource Server must still be able to distinguish CUSTOMER, MANAGER, ADMIN, and ORDER_REFUND. Customize the JWT access token so it contains the authenticated user's roles/authorities in a dedicated claim such as:

```
{
  "sub": "alice",
```

```
"scope": "orders.read orders.write",
"authorities": ["ROLE_CUSTOMER"]
}
```

The exact claim name is your choice, but it must be consistent with the Resource Server configuration.

TASK 2

Convert the Previous Exercise into a Resource Server

Replace HTTP Basic authentication with bearer JWT authentication

Take the Order Management application from the previous exercise and change its authentication model. The API must no longer authenticate users itself.

2.1 Configure OAuth 2.0 Resource Server support

- Remove the in-memory user authentication and HTTP Basic configuration from the Order API.
- Add OAuth 2.0 Resource Server support with JWT validation.
- Configure the Resource Server with the Authorization Server issuer URI.
- Use the Authorization Server metadata/JWK information to validate token signatures and issuer information.

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:9000
```

2.2 Map token claims to Spring Security authorities

Configure JWT authentication conversion so the claim produced by the Authorization Server becomes Spring Security **GrantedAuthority** instances in the Resource Server.

- A token containing ROLE_ADMIN must allow checks such as **hasRole("ADMIN")**.
- A token containing ORDER_REFUND must allow checks such as **hasAuthority("ORDER_REFUND")**.
- OAuth scopes should remain available as authorities as well (normally using the SCOPE_ prefix).

2.3 Preserve the previous endpoint-level rules

Reimplement the HTTP authorization rules from the previous exercise in the Resource Server configuration.

Endpoint	Expected rule
GET /api/products	Public - no token required
/api/admin/**	ADMIN only
POST /api/orders	Authenticated user required
Other /api/** endpoints	Authenticated user required
Anything else	Deny

2.4 Preserve the previous method-level rules

Method security must continue to work after the application becomes a Resource Server.

- **cancelAnyOrder()** - MANAGER or ADMIN.
- **refundOrder()** - requires ORDER_REFUND authority.
- **getOrder()** - order owner or ADMIN.
- **updateOrderStatus()** - preserve the manager/admin status rules from the previous exercise.

Validation and testing

Demonstrate the complete flow end to end.

1	Start both applications Authorization Server and Order Resource Server run independently on different ports.
2	Authenticate through the Authorization Server Use the registered client and one of the database-backed users.
3	Obtain an access token Complete the <code>authorization_code</code> flow and obtain a signed JWT.
4	Inspect the JWT Verify subject, scopes, issuer, expiry, and the custom roles/authorities claim.
5	Call the Resource Server Send Authorization: Bearer when calling protected Order API endpoints.
6	Verify authorization Show that users receive different results depending on roles, authorities, ownership, and endpoint rules.

Minimum acceptance checks

- ☐ Users are loaded from the Authorization Server database, not from an in-memory user store.
- ☐ Registered client details survive an Authorization Server restart.
- ☐ A valid Authorization Server token is accepted by the Order API.
- ☐ A request without a token to a protected endpoint returns 401.
- ☐ A valid token without sufficient authority returns 403 where appropriate.
- ☐ alice can access her own order but not another customer's order.
- ☐ admin can access administrative functionality.
- ☐ bob can invoke the refund operation because the token contains `ORDER_REFUND`.
- ☐ The public product endpoint still works without a token.

Bonus challenge

Persist OAuth 2.0 authorizations and consents as well, so authorization state and granted consent survive server restarts. Then add a second registered client with a different set of allowed scopes and demonstrate that the same user can receive different effective permissions depending on which client requests the token.